



Gráficos de barras

Los gráficos de barras se utilizan para mostrar datos categóricos o cuantitativos discretos, con barras rectangulares de longitudes proporcionales a los valores que representan. Un eje del gráfico (usualmente el **X**) muestra las categorías comparadas, y el otro eje (usualmente el **Y**) representa el valor medido. Estos valores pueden ser:

1. El total, promedio u otra medida de resumen de cada categoría
2. El conteo o frecuencia de cada categoría

Para ejemplificar el primer uso, grafiquemos el *dataframe* obtenido en la sección anterior:

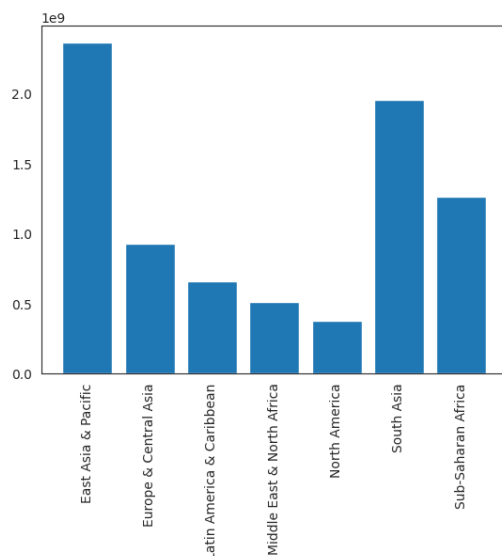
```
region_population = merged_df[['Region', '2023']].  
groupby('Region').sum()
```



2023	
Region	
East Asia & Pacific	2361090328
Europe & Central Asia	924969865
Latin America & Caribbean	657611624
Middle East & North Africa	508311359
North America	375077354
South Asia	1951539835
Sub-Saharan Africa	1259902353

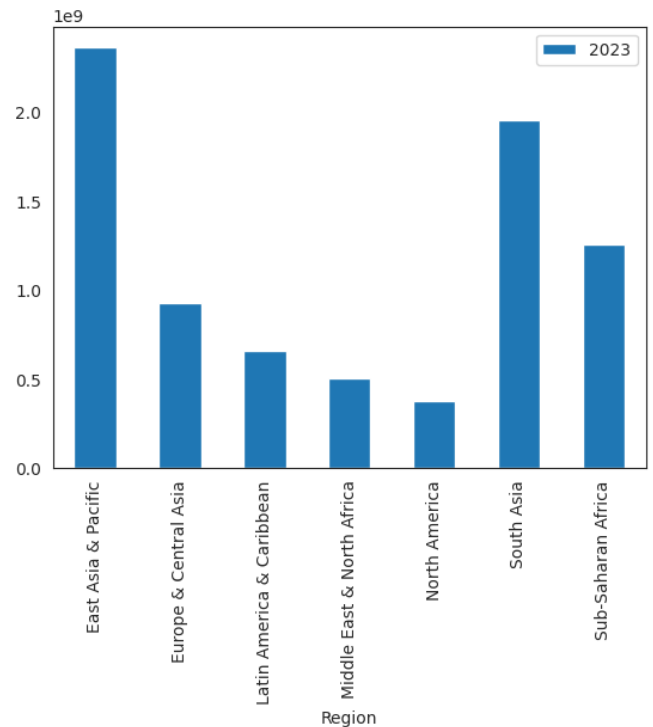
Con matplotlib

```
plt.bar(region_  
population.index, region_  
population['2023'])  
plt.xticks(rotation=90)
```



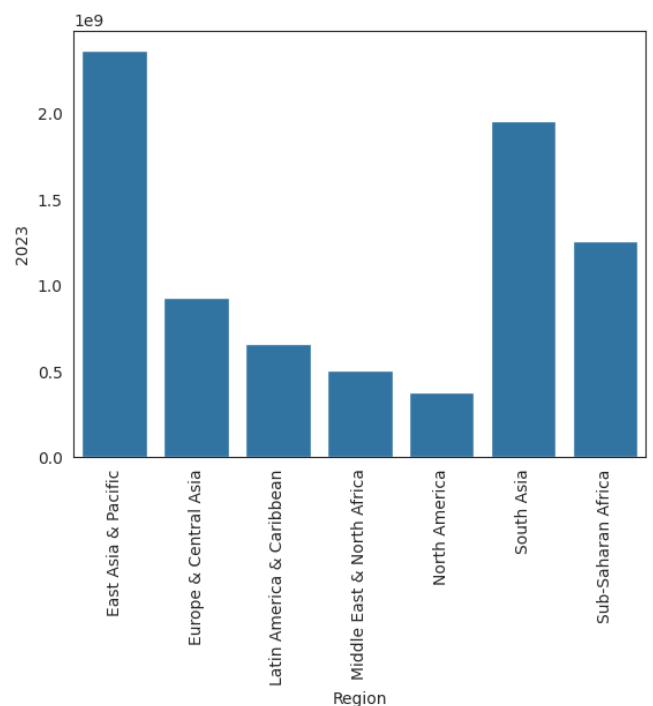
Con pandas

```
region_population.  
plot(kind='bar')
```



Con seaborn

```
sns.barplot(x=region_  
population.index, y=region_  
population['2023'])  
plt.xticks(rotation=90)
```



De los códigos anteriores puedes observar que, tanto en matplotlib como en seaborn, fue necesario rotar las etiquetas del eje X utilizando `xticks(rotation=90)`, ya que, de lo contrario, los textos se solapan y dificultan la lectura.

Para hacer evidente la ventaja estética de seaborn, veamos cómo cambiar los colores a las barras. Para ello, seaborn ofrece una forma sencilla mediante los parámetros `hue` y `palette`. Al especificar una columna en `hue` (que puede ser en principio la misma variable que en X), seaborn divide las barras por categoría y les asigna automáticamente colores distintos según la paleta indicada. En cambio, con matplotlib y pandas, se requiere generar manualmente una lista de colores recorriendo una paleta (obtenida con `plt.get_cmap()`) y pasarla como argumento al graficar. Así, si queremos emplear, por ejemplo, la paleta **viridis**, tendríamos que hacer:

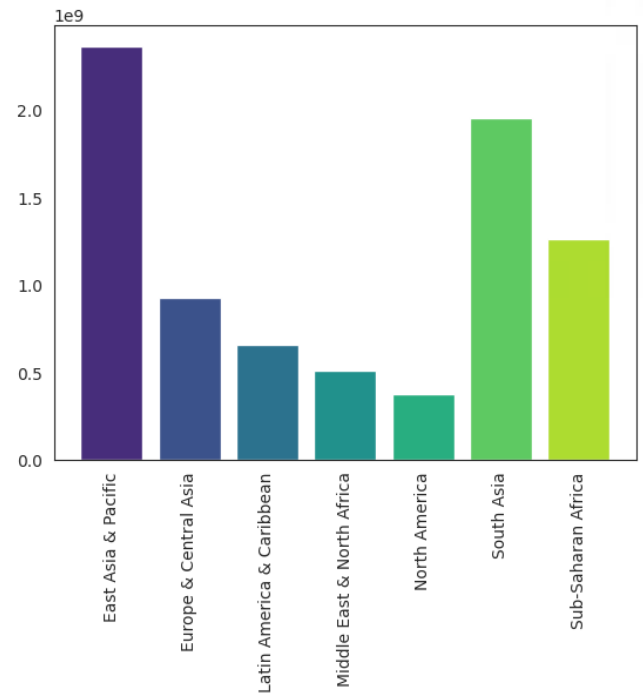
```
colors = sns.color_palette('viridis', n_colors=len(region_population))
```

Para cada índice i en el rango de las filas del *dataframe*, se normaliza el valor de i dividiéndolo por el número total de elementos, lo que da como resultado un valor entre 0 y 1. Este valor se pasa al **colormap**, que devuelve un color correspondiente a esa posición en el gradiente. De esta manera, se asigna un color único para cada barra en el gráfico.



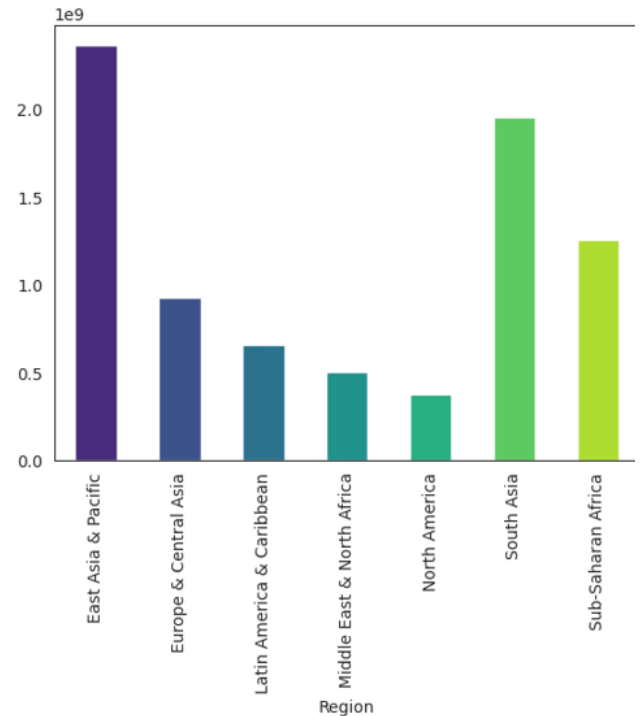
Con matplotlib

```
colors = sns.color_palette('viridis', n_colors=len(region_population))
plt.bar(region_population.index, region_population['2023'], color = colors)
plt.xticks(rotation=90)
```



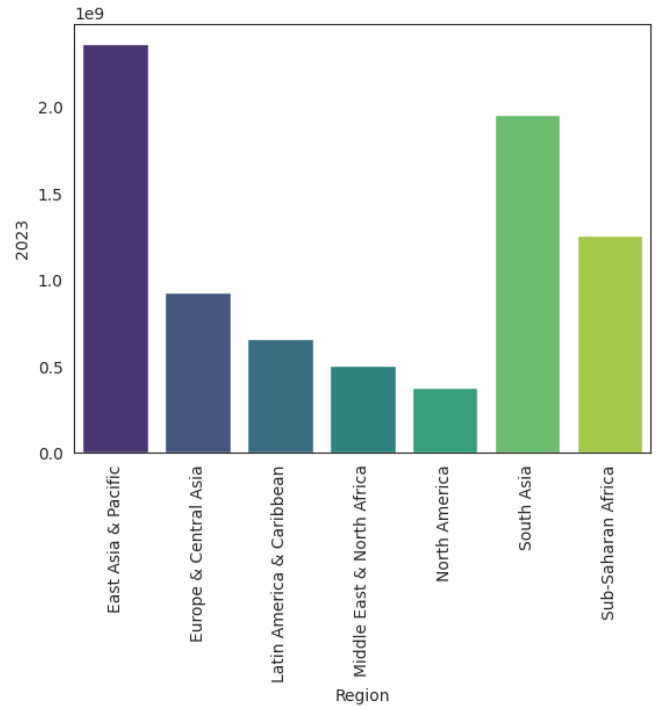
Con pandas

```
colors = sns.color_palette('viridis', n_colors=len(region_population))
region_population['2023'].plot(kind='bar', color=colors)
```



Con seaborn

```
sns.barplot(x=region_
population.index, y=region_
population['2023'],
hue=region_population.index,
palette='viridis')
plt.xticks(rotation=90)
```

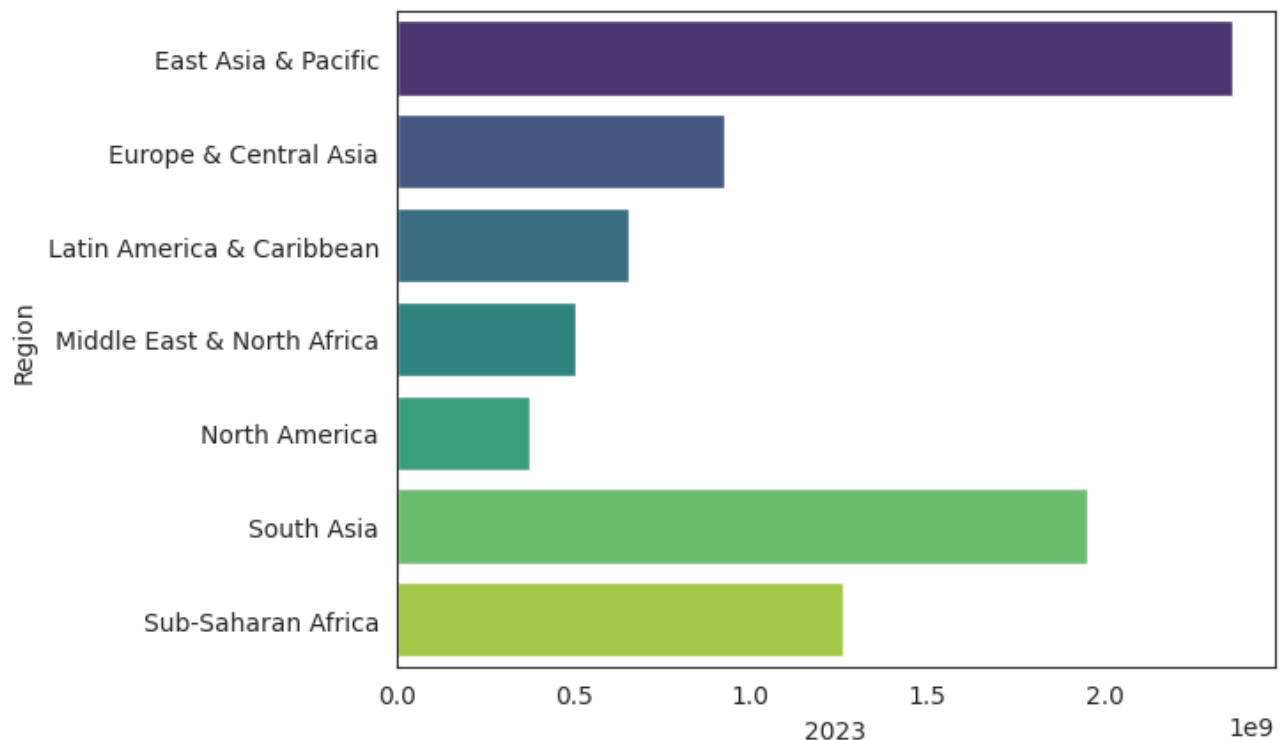


También puedes hacer gráficos de barras horizontales en las tres plataformas:

- Matplotlib: Con la función - `barh()`
- Pandas: Indicándolo en el tipo - `plot(kind='barh')`
- Seaborn: Incluyendo el parámetro `orient` - `barplot(orient='h')`

```
sns.barplot(y=region_population.index, x=region_
population['2023'], hue=region_population.index,
palette='viridis', orient='h')
```





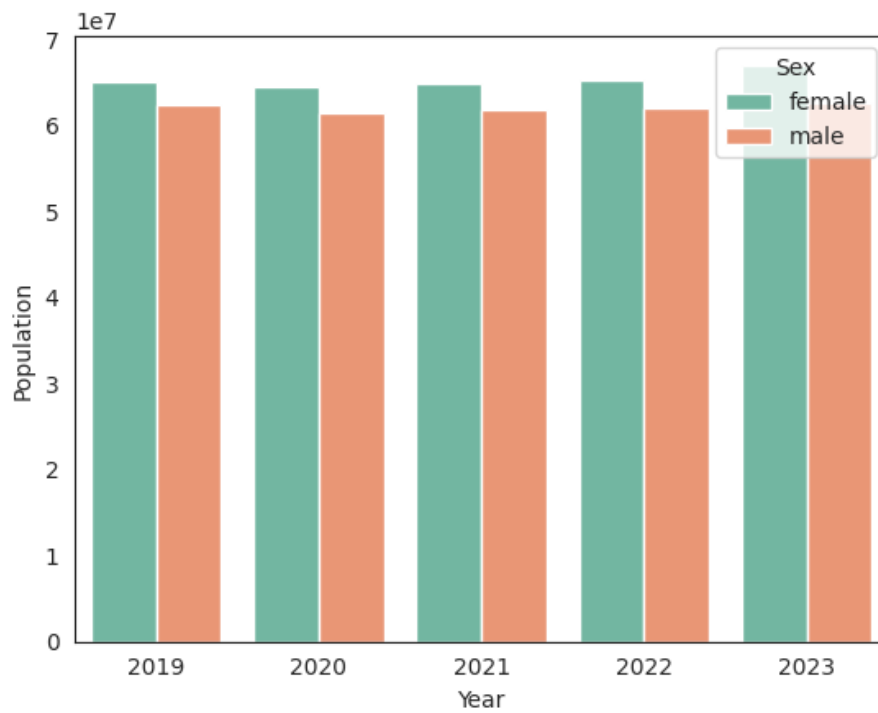
Nota: Al utilizar el parámetro `orient` en `seaborn`, es fundamental intercambiar los valores de los ejes **X** y **Y** para obtener el mismo resultado visual. En las otras plataformas esto no es necesario, ya que utilizan funciones de graficación distintas para ajustar la orientación de las barras.

Como recordarás de la lección anterior, el parámetro `hue` también se emplea para realizar agrupaciones semánticas. Retomando el ejemplo de la división por sexo para la población de México, podríamos hacer:

```
sns.barplot(x='Year', y='Population', hue='Sex', data=gender_mex)
```



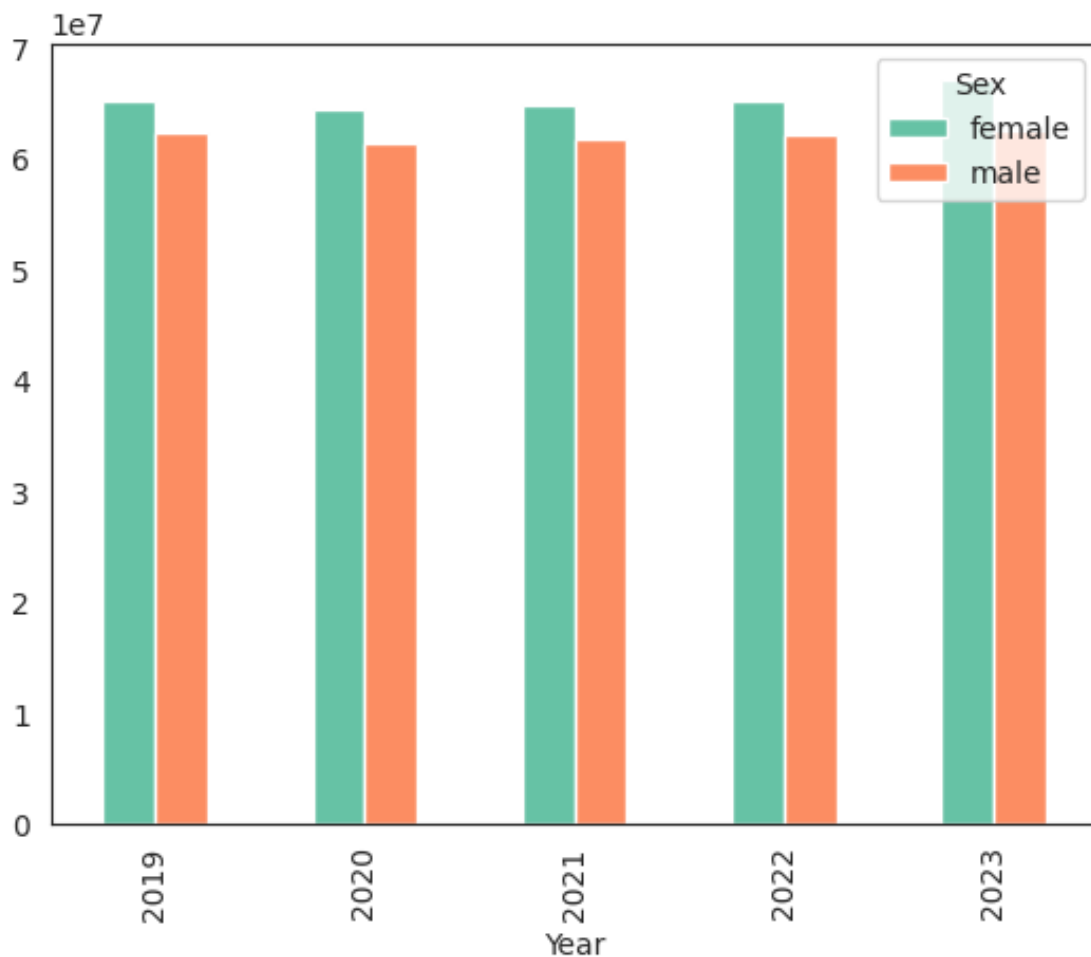
	Year	Sex	Population
0	2019	female	65172136
1	2019	male	62403393
2	2020	female	64540634
3	2020	male	61473390
4	2021	female	64912365
5	2021	male	61792773
6	2022	female	65345824
7	2022	male	62158301
8	2023	female	67075455
9	2023	male	62664304



Se empleó la paleta *Set2*: `sns.set_palette('Set2')`

Para lograr el mismo comportamiento con pandas, tendríamos que reestructurar el *dataframe*:

```
pivot_df = gender_mex.pivot(index='Year', columns='Sex',  
values='Population')  
pivot_df.plot(kind='bar')
```



Con matplotlib, además de la reestructuración anterior, se deben dibujar las barras de manera manual, separando los datos por categoría y trazando cada serie por separado:

```
pivot_df = gender_mex.pivot(index='Year', columns='Sex',  
values='Population')  
x = np.arange(len(pivot_df.index)) # Posición de los grupos  
(años)  
width = 0.35 # Ancho de cada barra  
plt.bar(x - width/2, pivot_df['female'], width,  
label='female')  
plt.bar(x + width/2, pivot_df['male'], width, label='male')  
plt.xticks(ticks=x, labels=pivot_df.index)  
plt.legend(['female', 'male'])
```



Recuerda que, cuando necesites hacer una separación semántica por una variable categórica, seaborn ofrece una sintaxis mucho más sencilla y directa.

El segundo uso de las gráficas de barras es el conteo o frecuencia de cada categoría. Por ejemplo, si queremos saber cuántos países hay por grupo de ingresos en el indicador de población, basta con contar cuántas veces aparece cada grupo de ingresos en el *dataframe*. Para ello puedes emplear nuevamente el *dataframe* `metadata`:



	Country	Code	Region	IncomeGroup
0	ABW	Latin America & Caribbean		High income
1	AFE		NaN	Low income
2	AFG		South Asia	Low income
3	AFW		NaN	Low income
4	AGO	Sub-Saharan Africa		Lower middle income
...	...		...	...
260	XKX	Europe & Central Asia		Upper middle income
261	YEM	Middle East & North Africa		Low income
262	ZAF	Sub-Saharan Africa		Upper middle income
263	ZMB	Sub-Saharan Africa		Lower middle income
264	ZWE	Sub-Saharan Africa		Lower middle income

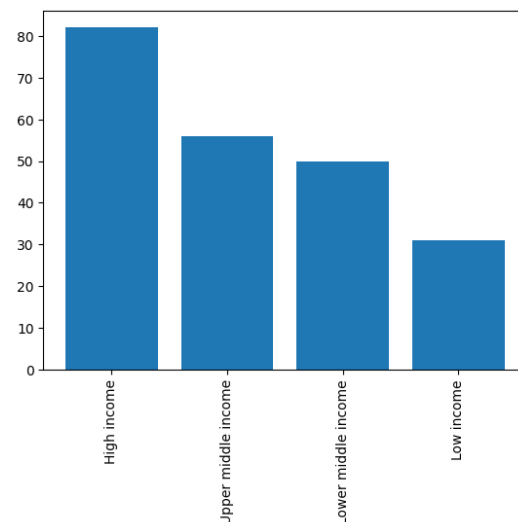
265 rows × 3 columns

En el caso de matplotlib y pandas, para generar el gráfico tenemos que auxiliarnos de la función `value_counts()`. Seaborn, por su parte, simplifica este proceso al ofrecer un gráfico diseñado específicamente para este propósito: `countplot`, lo que representa una ventaja significativa en términos de practicidad y legibilidad del código.



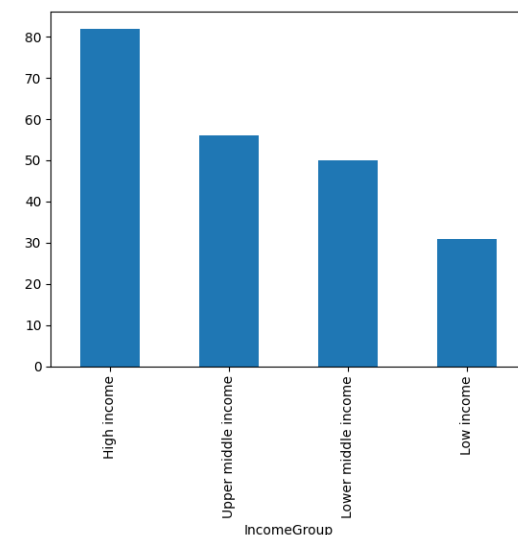
Con matplotlib

```
counts = metadata['IncomeGroup'].  
value_counts()  
plt.bar(counts.index, counts.values)  
plt.xticks(rotation=90)
```



Con pandas

```
metadata['IncomeGroup'].value_  
counts().plot(kind='bar')
```

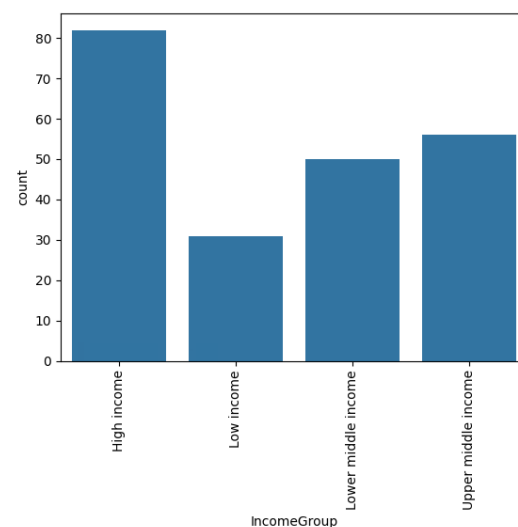


Con seaborn

```
sns.countplot(x='IncomeGroup',  
data=metadata)  
plt.xticks(rotation=90)
```

Para ordenar los resultados como en las otras plataformas emplea el parámetro `order`:

```
order = metadata['IncomeGroup'].  
value_counts().index
```



Después de haber explorado los gráficos de barras, que son ideales para comparar cantidades entre categorías, es conveniente recurrir a los gráficos circulares cuando las cantidades representan partes de un total. Este tipo de gráfico facilita la visualización de cómo se distribuyen esas partes dentro del conjunto, permitiendo una comprensión rápida de las proporciones relativas.

| Antes de terminar...

Los gráficos de barras son una herramienta visual poderosa para comparar cantidades entre diferentes categorías. Su simplicidad y claridad los hacen ideales para identificar diferencias, resaltar tendencias y comunicar datos categóricos de forma efectiva. Además, su versatilidad permite personalizaciones que enriquecen el análisis, como el uso de colores y agrupamientos.

De las tres plataformas, seaborn es la que permite aplicar colores de forma más sencilla y automática, especialmente cuando se utiliza el parámetro `hue` junto con una paleta. Esto facilita la creación de gráficos visualmente atractivos sin necesidad de definir manualmente los colores para cada categoría. Además, seaborn también facilita la agrupación semántica de los datos, permitiendo representar subcategorías de manera clara dentro de un mismo gráfico, lo que enriquece la interpretación sin complicar la codificación.

Cada vez que tengas que construir un gráfico de barras a partir de tu información tabular, pregúntate lo siguiente:

- ¿Qué tipo de datos vas a representar en las barras?, ¿corresponden a categorías (variables cualitativas)?
- ¿El gráfico mostrará únicamente una variable (univariado), como la cantidad por categoría, o quieres comparar una segunda variable dentro de cada categoría (bivariado)?
- ¿Te interesa visualizar la frecuencia de cada categoría o vas a representar medidas ya calculadas (suma, promedio, etc.)?

Material adicional

● Matplotlib

The Matplotlib development team. (2025). *Matplotlib: Visualization with Python* [Matplotlib: Visualización con Python]. Matplotlib. Recuperado el 25 de abril de 2025 de: <https://matplotlib.org>

● Pandas

Pandas via. (2025). *Pandas*. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org>

● Seaborn

Waskom, M. (2024). *Seaborn: statistical data visualization* [Seaborn: visualización de datos estadísticos]. Seaborn. Recuperado el 25 de abril de 2025 de: <https://seaborn.pydata.org>

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.

Glosario

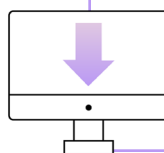
- **Gráficos de barras:** Se utilizan para mostrar datos categóricos o cuantitativos discretos, con barras rectangulares de longitudes proporcionales a los valores que representan.

Bibliografía

Melo, M. (26 de enero de 2024). *2023: aumento de llegadas de migrantes a México en ruta hacia EE. UU.* Statista. Recuperado el 25 de abril de 2025 de: <https://es.statista.com/grafico/16470/migrantes-registrados-y-devueltos-en-mexico/>

The World Bank Group. (2025). *Indicators* [Indicadores]. World Bank Group. Recuperado el 25 de abril de 2025 de: <https://data.worldbank.org/indicator>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.